

k-SEC + DBVF: A Technical Reference

Cubic-Equivariant k-Space Convolution and a Differentiable Bond-Valence Field for
Solid-State Electrolyte Conductivity Prediction

Sunwoo Lee

2026

Contents

Front matter	2
Symbol glossary	4
Part A — Structure (architecture)	5
A.0 Pipeline overview	5
A.1 Featurization to input tensors	6
A.2 Atom embedding	7
A.3 Reciprocal-space construction	7
A.4 KSECBLOCK — the encoder unit	8
A.4.1 KubicHarmonicFilter	8
A.4.2 Complex magnitude gate	8
A.4.3 CrossShellGatedAttention	10
A.4.4 Complex layer norms (<code>_CLN</code>)	10
A.5 Magpie composition projection	10
A.6 Lattice projection	10
A.7 Geometric projection	11
A.8 Differentiable Bond-Valence Field (DBVF)	11
A.8.1 LearnableBVParams — the only trainable physics parameters	11
A.8.2 <code>compute_bv_features</code> — the bond-valence field evaluation	12
A.8.3 BV projection head	13
A.9 Concatenation, readout MLP, and target shift	13
A.10 Component table — verified parameter counts	13
A.11 Equivariance and invariance, summarised	14
A.12 Optional / inactive components (consolidated brief)	14
Part B — Mechanism (mathematical formulation)	16
B.0 Notation reminder	16
B.1 Featurization equations	16
B.2 The reciprocal-space transform	16
B.3 Cubic-rotation invariants of $\mathbf{k}$	16
B.4 Kubic-harmonic filter	17

B.5 Cross-shell gated attention	17
B.6 KSECBLOCK forward	18
B.7 DBVF forward	18
B.8 Auxiliary projection heads	19
B.9 Concatenation and readout	19
B.10 Target shift	19
B.11 Loss	20
B.12 Optimisation	20
B.13 Cross-validation protocol	20
B.14 Pretraining (encoder only)	20
B.15 Stacking with a tabular gradient-boosted-tree baseline	21
Part C — Reasoning (design justification)	22
C.1 Why reciprocal space?	22
C.2 Why O_h cubic invariants specifically?	22
C.3 Why cross-shell gated attention?	23
C.4 Why bond-valence as inductive bias for σ ?	23
C.5 Why aggregate BV mismatch statistics, not per-site features?	23
C.6 Why fuse Magpie / Lattice / Geometric / DBVF / k-SEC streams?	23
C.7 Why pretrain the encoder on Materials Project crystals?	24
C.8 Why MSE loss + cosine-annealed AdamW?	24
C.9 Why stack with LightGBM?	24
C.10 Component complementarity	24
C.11 Honest negative result: DBVF features vs. DBVF architecture	25
Verification	26
How to confirm the documented forward path	26
What changes (and what doesn't) when flags toggle	26
References (key code paths)	27

Front matter

This document is a self-contained technical reference for the model published as:

Cubic-Equivariant k-Space Convolution and a Differentiable Bond-Valence Field for Ionic-Conductivity Prediction in Solid-State Electrolytes. Sunwoo Lee, 2026. Submitted to *Journal of Materials Science & Technology* (manuscript ID J-MST-D-26-03318); preprint at ChemRxiv DOI [10.26434/chemrxiv.15002591/v1](https://doi.org/10.26434/chemrxiv.15002591/v1); code at github.com/snoco53/k-sec-dbfv.

The reference covers **the headline forward pass** — the model invoked by `scripts/08_train_hybrid.py` with flags `--use-bv-field --use-lattice --use-geometric` against the OBELiX benchmark of 281 solid-state lithium-ion electrolytes, producing the reported standalone mean absolute error (MAE) of **1.047** on $\log_{10} \sigma$. Optional components present in the codebase but not active in the headline (Path Bond-Valence Field, the BatteryNet dual-stream MPNN, MACE features, heteroscedastic NLL, MC dropout) are summarised in §A.12 and not given full per-component treatment.

Assumed background. Linear algebra, basic deep learning (multi-layer perceptron, dropout, layer normalisation, attention), and undergraduate solid-state chemistry. Reciprocal-lattice basics, cubic point-group invariants, and bond-valence theory are introduced briefly when first needed.

Audience-facing terminology. All abbreviations are spelt out at first use:

- **MLP** — multi-layer perceptron
- **GNN** — graph neural network
- **MAE** — mean absolute error
- **MP** — Materials Project (the open materials-informatics database)
- **AIMD** — *ab initio* molecular dynamics
- **SSE** — solid-state electrolyte
- **CIF** — Crystallographic Information File
- **k-SEC** — k-Space Equivariant Convolution
- **DBVF** — Differentiable Bond-Valence Field
- **BVS** — bond-valence sum (Brown’s classical theory)

License. Code is released under the MIT licence; this reference document is provided alongside the open-source release at github.com/snoo53/k-sec-dbvfn.

Symbol glossary

Symbol	Meaning	First defined
N	total atoms across a batch	§A.1
B	batch size (number of crystals)	§A.1
K	number of reciprocal-space points; $K = 1241$ at $n_{\max} = 2$	§A.4
D	k-space hidden dimension; $D = 96$ in headline	§A.4
$\mathbf{r}_n \in [0, 1)^3$	fractional coordinate of atom n	§A.1
$\mathbf{x}_n \in \mathbb{R}^3$	Cartesian coordinate of atom n	§B.7
$z_n \in \{1, \dots, 100\}$	atomic number of atom n	§A.1
$\mathbf{C} \in \mathbb{R}^{3 \times 3}$	per-crystal lattice matrix (rows are basis vectors, in Å)	§A.1
$\mathbf{e}_n \in \mathbb{R}^D$	learned atom embedding of z_n	§A.3
$\mathbf{k}_m \in \mathbb{R}^3$	m -th reciprocal-lattice integer vector (Wyckoff orbit member)	§A.4
$ k $	$\ \mathbf{k}_m\ $, magnitude of $\mathbf{k}_m$	§A.4
$\hat{\mathbf{k}} = \mathbf{k}/ k $	unit direction vector	§B.3
$K_0, K_{4a}, K_{4b}, K_{6a}, K_{6b}$	five cubic-rotation-group (O_h) invariant polynomials of $\hat{\mathbf{k}}$	§B.3
$\mathbf{F}_b(\mathbf{k}_m) \in \mathbb{C}^D$	structure factor: discrete Fourier transform of embedded atoms in crystal b	§B.2
$\mathbf{H} \in \mathbb{C}^{B \times K \times D}$	k-space feature tensor	§A.4
σ	room-temperature ionic conductivity in S cm^{-1}	§A.0
$\log_{10} \sigma$	base-10 log of σ , the regression target	§A.0
V_i	bond-valence sum at Li site i	§B.7
r_0, b	learned bond-valence parameters per anion species	§B.7
$V_{\text{target}} = 1$	nominal Li valence used as the BVS target	§B.7
$U_i = V_i - V_{\text{target}} $	bond-valence mismatch at Li site i	§B.7
$\hat{y}$	model’s predicted $\log_{10} \sigma$	§A.10
s	scalar <code>log_sigma_shift</code> parameter, set per fold	§A.10
$\mathcal{L}$	training loss (MSE on $\log_{10} \sigma$)	§B.11

Part A — Structure (architecture)

A.0 Pipeline overview

The headline model maps a crystal structure (atomic numbers, fractional coordinates, lattice matrix, plus three pre-computed feature vectors) to a single scalar $\log_{10} \sigma$ prediction. Forward pass at a glance:

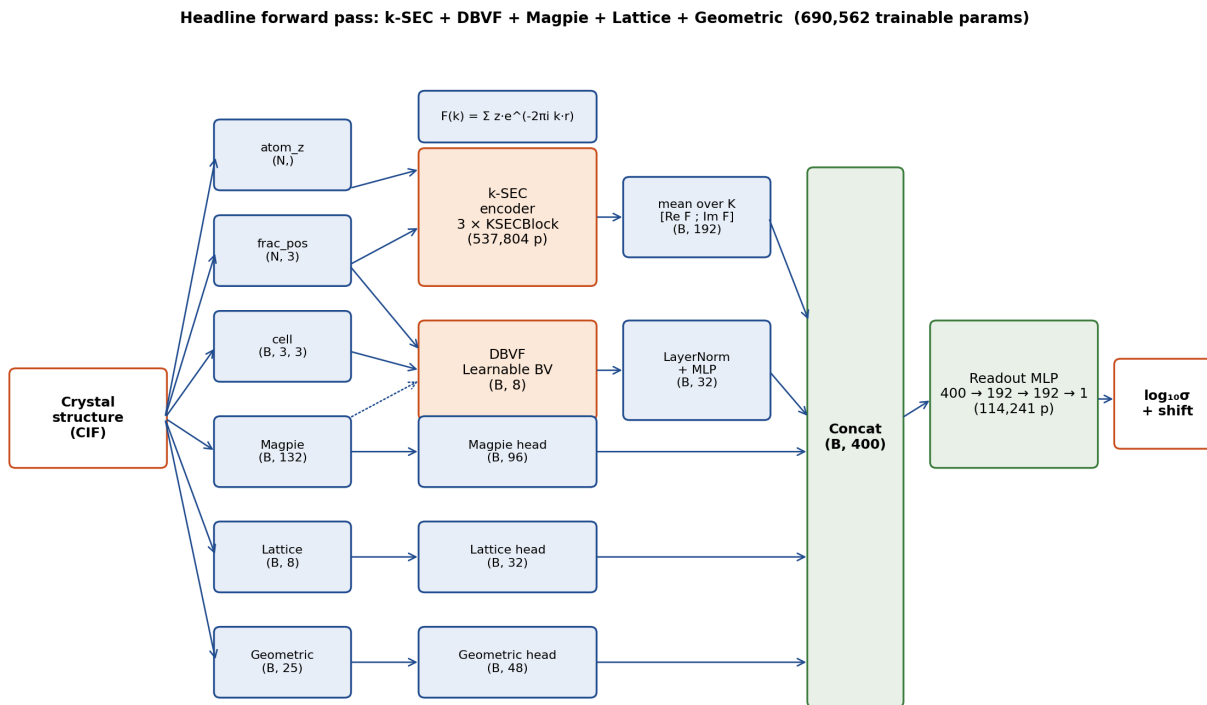


Figure 1: Headline forward pass. The Concat box concatenates six branches into a 400-dimensional vector that the readout MLP maps to a scalar prediction; the per-fold target shift s is added at the end. Parameter counts shown are verified by `sum(p.numel() for p in m.parameters())`.

The model has **690,562 trainable parameters** in this configuration. The forward pass has six concatenation branches into the readout MLP:

1. Real and imaginary parts of the mean-pooled k-space feature tensor ($96 + 96 = 192$ dim).
2. Magpie composition projection (96 dim).
3. Lattice-matrix projection (32 dim).
4. Geometric (BV-physics) projection (48 dim).
5. DBVF projection (32 dim).
6. — *None*. Total: $192 + 96 + 32 + 48 + 32 = 400$ dim into the readout.

The codebase entry point is `src/ionpath/models/kspace_conv.py:213` (KSENet). The training entry point is `scripts/08_train_hybrid.py:307` (model construction) and `:336` (training loop).

A.1 Featurization to input tensors

A single crystal in OBELiX is parsed from a Crystallographic Information File (CIF) via `pymatgen.core.Structure` and exposed as a `CrystalGraph` dataclass (`src/ionpath/data/schema.py`). Featurization is performed once and cached; the model consumes the cached tensors at training time.

Per-atom tensors (concatenated across the batch):

- `atom_z` : (N,) `int64` — atomic numbers, computed at `src/ionpath/data/featurize.py:70` from `site.specie.symbol`. Hydrogen $\rightarrow$ 1, Lithium $\rightarrow$ 3, etc., supporting elements 1–92.
- `frac_pos` : (N, 3) `float32` — fractional coordinates modulo 1.0, `featurize.py:71`.
- `batch_idx` : (N,) `int64` — per-atom index telling the model which crystal each atom belongs to.

Per-crystal tensors:

- `cell` : (B, 3, 3) `float32` — the lattice matrix in Ångströms; rows are the three basis vectors. `featurize.py:72`.
- `magpie` : (B, 132) `float32` — composition descriptor from Ward et al.’s “Magpie” set (`matminer.featurizers.composition.ElementProperty.from_preset("magpie")`). 132-d real vector summarising atomic-property statistics over the formula. Built at `src/ionpath/data/magpie.py:48–58`.
- `lattice_feats` : (B, 8) `float32` — eight scalars derived from the `Lattice` object, in order: a, b, c (edge lengths in Å), $\alpha_{\text{rad}}, \beta_{\text{rad}}, \gamma_{\text{rad}}$ (inter-axis angles in radians), V (volume in Å³), N/V (atomic number density in atoms/Å³). `featurize.py:78–85`.
- `geometric` : (B, 25) `float32` — physics-motivated descriptors of the lithium environment: Li coordination statistics, Li–Li connectivity (including a percolation-dimension estimate), framework composition, bottleneck radii at Li–Li midpoints, and a bond-valence strain proxy. Computed in `src/ionpath/data/geometric.py:85–277`. Index 24 is a binary `feat_valid` flag that is 1.0 when computation succeeded.

The full 25-d geometric vector (in order) is:

Index	Name	Description
0–4	Li coordination	<code>cn_mean</code> , <code>cn_std</code> , <code>neighbor_d_mean</code> , <code>neighbor_d_min</code> , <code>neighbor_d_std</code> — Li–anion coordination numbers and bond distances within 3.5 Å
5–9	Li–Li connectivity	<code>li_li_min</code> , <code>li_li_median</code> , <code>li_li_max</code> , <code>li_li_count_within_4Å</code> , <code>li_volume_per_li</code> — per-Li volume is the inverse density

Index	Name	Description
10–14	Framework	density of non-Li atoms, fractions of Li / anion / transition metal, mean nominal anion charge
15–18	Cell	volume_per_atom, n_atoms, n_li, aspect_ratio
19–23	BV / percolation	bottleneck_radius_min, bottleneck_radius_mean, li_percolation_3D, li_percolation_dim, bv_strain_proxy
24	Validity	1.0 if all sub-computations succeeded

A.2 Atom embedding

src/ionpath/models/kspace_conv.py:259:

```
self.embed = nn.Embedding(num_species, feature_dim)
```

With num_species=100, feature_dim=96, the embedding is a 100×96 table $\rightarrow$ **9,600 parameters**. It maps atom_z : (N,) int $\rightarrow$ e : (N, 96) float. The atomic-number index 0 is reserved for “unknown element”; valid atomic numbers index 1–92.

A.3 Reciprocal-space construction

The encoder operates on the discrete Fourier transform of the embedded atomic field, evaluated on a finite reciprocal-lattice grid that respects cubic-rotation symmetry.

Grid construction. At kspace_conv.py:267–283 the KSECNet constructor builds the wavevector grid by:

1. Calling generate_wyckoff_wavevectors(n_max=2) (src/ionpath/utils/wyckoff_fourier.py:75–103), which enumerates integer triples (n_x, n_y, n_z) with $|n_i| \leq n_{\max}$ and groups them into orbits under the 24 proper rotations of the cubic point group O_h . At $n_{\max} = 2$ this yields **70 orbit representatives**.
2. Calling precompute_orbits(...) (wyckoff_fourier.py:173–190) to expand each representative back into all members of its orbit, returning a list of tensors.
3. Concatenating every orbit member, prepending the Γ -point (0,0,0), yielding $K = 1,241$ k-points total.

The full $\mathbf{k}_m$ grid, the magnitudes $|k_m|$, and the per-grid Kubic-invariant evaluations $K_0, K_{4a}, K_{4b}, K_{6a}, K_{6b}$ (zeroed at Γ since it has no direction; kspace_conv.py:282) are stored as buffers — they are not learned.

Structure factor. At `kspace_conv.py:412-426`, after embedding the atoms, the model computes the (un-normalised) discrete Fourier transform per crystal b :

$$\mathbf{F}_b(\mathbf{k}_m) = \frac{1}{N_b} \sum_{n \in b} \mathbf{e}_n e^{-2\pi i \mathbf{k}_m \cdot \mathbf{r}_n}$$

producing $\mathbf{F} : (B, K, D)$ as a complex tensor. Concretely (`kspace_conv.py:416-426`): phases are $\phi = -2\pi(\text{frac_pos} \cdot \mathbf{k}_m^\top)$; the complex exponential is constructed as `complex(cos(phi), sin(phi))`; per-atom contributions are summed via `index_add_` indexed by `batch_idx`; finally each crystal is normalised by its atom count.

This step has **0 trainable parameters** — it is a deterministic transform, but it is differentiable because gradients flow back through the embedding (and, via the structure factor, through `frac_pos` if the user enables Hessian-style probes elsewhere).

A.4 KSECBLOCK — the encoder unit

The k-space tensor $\mathbf{H} = \mathbf{F}$ passes through `num_blocks = 3` identical KSECBLOCK units (`kspace_conv.py:429-430`). Each block contains a Kubic-harmonic filter, a complex-magnitude self-gate, and cross-shell gated attention, with two complex layer-norms. Per block: **179,268 parameters**. Three blocks: **537,804 parameters**.

A.4.1 KubicHarmonicFilter

`KubicHarmonicFilter` at `kspace_conv.py:79-115` is an MLP-parameterised pointwise complex filter $\mathbf{W}(\mathbf{k}) \odot \mathbf{H} + \mathbf{b}(\mathbf{k})$ where the weight $\mathbf{W}$ and bias $\mathbf{b}$ depend on $\mathbf{k}$ only through the six O_h -invariant scalars $[|k|, K_0, K_{4a}, K_{4b}, K_{6a}, K_{6b}]$.

- `gain_mlp` is a 3-layer MLP $6 \rightarrow 64 \rightarrow 64 \rightarrow 192$ producing the real and imaginary parts of $\mathbf{W}$ ($D = 96$, so $2D = 192$). With biases: 17,088 parameters.
- `bias_mlp` is a 2-layer MLP $6 \rightarrow 64 \rightarrow 192$ producing the real and imaginary parts of $\mathbf{b}$. With biases: 12,928 parameters.
- Filter total per block: **30,016 parameters**.

The filter is *equivariant under O_h by construction* (§C.2): rotating $\mathbf{k}$ by any cubic-symmetry rotation leaves $|k|$ and the K -invariants unchanged, so the filter output is unchanged.

A.4.2 Complex magnitude gate

`kspace_conv.py:204-207`. After the Kubic filter, each k-mode is rescaled by a learned magnitude-dependent gate:

$$\mathbf{H} \leftarrow \mathbf{H} \cdot \sigma(|\mathbf{H}| - 1)$$

where σ is the logistic sigmoid and $|\mathbf{H}|$ is the elementwise complex magnitude. This is a deterministic non-linearity with no parameters. It contracts very small magnitudes toward zero and saturates large ones near 1, while being smooth and differentiable.

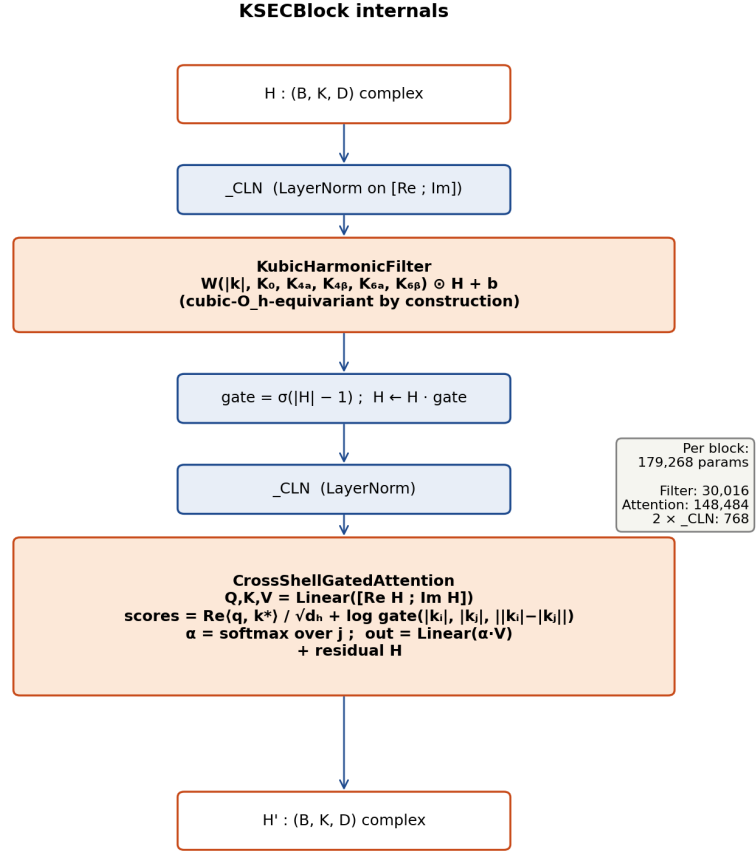


Figure 2: Structure of one KSECBLOCK.

A.4.3 CrossShellGatedAttention

CrossShellGatedAttention at `kspace_conv.py:118-175` is a 4-head Hermitian-style self-attention over the K k-space slots, gated by a learned per-edge function of the wave-vector magnitudes.

Per block:

- `qkv : Linear(2D, 6D) = Linear(192, 576)` projects the real-imag-stacked input into Q, K, V in real-imaginary form. With bias: 111,168 parameters.
- `o : Linear(2D, 2D) = Linear(192, 192)` is the output projection. With bias: 37,056 parameters.
- `gate_mlp` is a 2-layer MLP $3 \rightarrow 32 \rightarrow n_{\text{heads}} = 4$ producing per-head gate logits as a function of $(|k_i|, |k_j|, ||k_i| - |k_j||)$. 260 parameters.
- Attention total per block: **148,484 parameters**.

The attention scores are the real part of the Hermitian inner product $\langle q, k^* \rangle$ scaled by $\sqrt{d_h}$ (with $d_h = D/4 = 24$), then a $\log \text{gate}(|k_i|, |k_j|, \cdot)$ term is added to the score before softmax — equivalent to multiplying the un-normalised attention weight by $\text{gate} \in (0, 1)$. A residual connection adds **H** back to the attention output (`kspace_conv.py:175`).

A.4.4 Complex layer norms (_CLN)

`kspace_conv.py:178-187`. A `_CLN` is a `nn.LayerNorm(2D)` applied to the real-imaginary-concatenated tensor and then split back. Two `_CLNs` per block: $2 \times 192 \times 2 = 768$ parameters.

A.5 Magpie composition projection

`kspace_conv.py:291-296`:

```
self.tabular_norm = nn.LayerNorm(132)
self.tabular_proj = nn.Sequential(
    nn.Linear(132, 96), nn.SiLU(), nn.Dropout(0.15),
    nn.Linear(96, 96), nn.SiLU(),
)
```

Total: $264 + 12,768 + 9,312 = \mathbf{22,344}$ parameters. Consumes the (B, 132) Magpie composition vector and produces (B, 96) for concatenation with the k-space mean. Called at `kspace_conv.py:456-461`.

A.6 Lattice projection

`kspace_conv.py:303-308`:

```
self.lattice_norm = nn.LayerNorm(8)
self.lattice_proj = nn.Sequential(
    nn.Linear(8, 32), nn.SiLU(),
    nn.Linear(32, 32), nn.SiLU(),
)
```

Total: $16 + 288 + 1,056 = \mathbf{1,360}$ parameters. Consumes (B, 8) lattice scalars; produces (B, 32).

A.7 Geometric projection

kspace_conv.py:314-319:

```
self.geometric_norm = nn.LayerNorm(25)
self.geometric_proj = nn.Sequential(
    nn.Linear(25, 48), nn.SiLU(), nn.Dropout(0.15),
    nn.Linear(48, 48), nn.SiLU(),
)
```

Total: $50 + 1,248 + 2,352 = \mathbf{3,650}$ parameters. Consumes (B, 25) geometric features; produces (B, 48).

A.8 Differentiable Bond-Valence Field (DBVF)

bv_field=True activates DBVF at kspace_conv.py:336-351. DBVF has three sub-modules: learnable per-anion parameters, a deterministic feature-extraction function, and a projection head.

DBVF: differentiable bond-valence field, atom-level $\rightarrow$ per-crystal aggregates (202 learnable params)

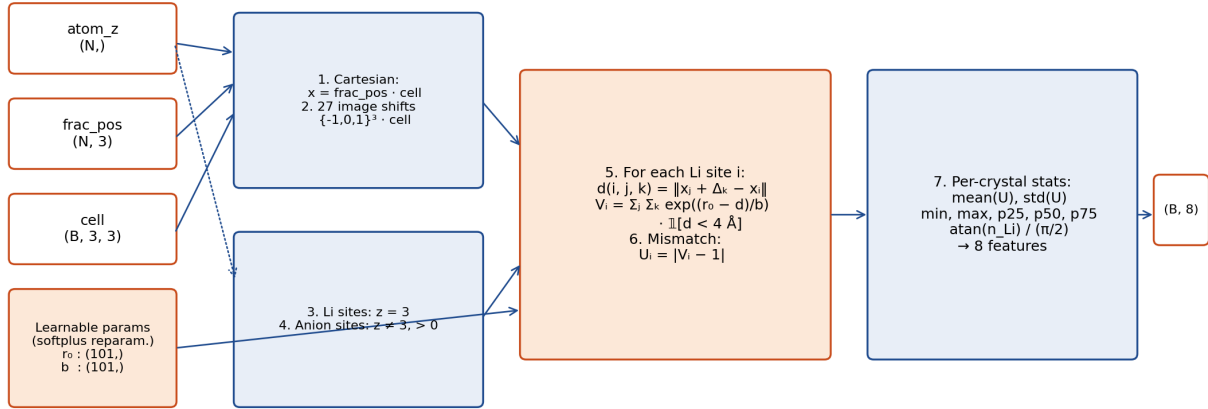


Figure 3: DBVF flow. Learnable parameters r_0, b enter at step 5 via softplus reparameterisation. Steps 1–4 are deterministic geometry. Step 5 evaluates the bond-valence sum at every Li site over a ± 1 image shell. Step 7 reduces the per-Li mismatch distribution to 8 per-crystal aggregates.

A.8.1 LearnableBVPparams — the only trainable physics parameters

src/ionpath/models/bond_valence_field.py:68-101. Two `nn.Parameter` vectors of shape $(\text{num_species} + 1,) = (101,)$ each:

- `r0_raw` : (101,) — softplus-reparameterised ($r_0 = \text{softplus}(r_0^{\text{raw}}) > 0$) — 101 params.

- `b_raw` : (101,) — softplus-reparameterised ($b = \text{softplus}(b^{\text{raw}}) > 0$) — 101 params.

Total: **202 parameters**.

Initialisation (`bond_valence_field.py:81-87`): every species is initialised to $(r_0, b) = (1.8, 0.4)$, then the seven known Li-anion pairs in the Brown 2002 review are overwritten from `_LI_BV_INIT` (`bond_valence_field.py:45-54`):

Anion	r_0 (Å)	b (Å)
O	1.466	0.37
S	1.85	0.40
Se	1.93	0.40
F	1.36	0.37
Cl	1.79	0.40
Br	1.92	0.40
I	2.07	0.40
N	1.61	0.37

These initial values are inverted through `_inv_softplus` (`bond_valence_field.py:91-93`) so that the softplus mapping recovers them at step zero of training.

A.8.2 `compute_bv_features` — the bond-valence field evaluation

`bond_valence_field.py:104-185`. Given the learnable parameters and the same `(atom_z, frac_pos, cell, batch_idx)` already consumed by k-SEC, the function returns a `(B, 8)` per-crystal feature tensor.

For each crystal b in the batch, the algorithm (`bond_valence_field.py:135-183`):

1. Builds Cartesian positions $\mathbf{x}_n = \mathbf{r}_n \cdot \mathbf{C}$ (line 127).
2. Constructs 27 image shifts $\Delta_k \in \{-1, 0, 1\}^3 \cdot \mathbf{C}$ (lines 130–155).
3. Masks Li sites ($z = 3$) and anion sites ($z \neq 3, z > 0$).
4. For each Li site i , computes the bond-valence sum over the cation–anion pairs within a 4 Å cutoff:

$$V_i = \sum_{j \in \text{anions}} \sum_{k \in \pm 1\text{-images}} \exp\left(\frac{r_0(z_j) - d_{ijk}}{b(z_j)}\right) \cdot \mathbb{1}[d_{ijk} < 4 \text{ Å}]$$

where $d_{ijk} = \|\mathbf{x}_j + \Delta_k - \mathbf{x}_i\|$ (lines 158–169).

5. The per-Li mismatch is $U_i = |V_i - 1|$; the target valence 1.0 is hard-coded.
6. The 8 per-crystal aggregate features are $[\text{mean}(U), \text{std}(U), \text{min}(U), \text{max}(U), p_{25}(U), p_{50}(U), p_{75}(U), \arctan(n_L)]$ (lines 172–183), where the last entry is a smooth saturating count of Li sites.

This sub-module has 0 trainable parameters of its own — it consumes `LearnableBVParams` but only as constants for that forward pass.

A.8.3 BV projection head

kspace_conv.py:341-346:

```
self.bv_norm = nn.LayerNorm(8)
self.bv_proj = nn.Sequential(
    nn.Linear(8, 32), nn.SiLU(),
    nn.Linear(32, 32), nn.SiLU(),
)
```

Total: $16 + 288 + 1,056 = \mathbf{1,360}$ parameters. Consumes (B, 8) aggregates; produces (B, 32). Called at kspace_conv.py:484-493. **DBVF total** (params + projection): $202 + 1,360 = 1,562$ parameters.

A.9 Concatenation, readout MLP, and target shift

After all branches are computed, the model concatenates them (kspace_conv.py:451-493) into a (B, 400) vector $\mathbf{h}$:

$$\mathbf{h} = [\text{Re } \bar{\mathbf{F}} \parallel \text{Im } \bar{\mathbf{F}} \parallel \text{Magpie} \parallel \text{Lattice} \parallel \text{Geometric} \parallel \text{DBVF}]$$

where $\bar{\mathbf{F}}$ is the mean of $\mathbf{H}$ over the K axis (kspace_conv.py:452-453). The dimensions are $96 + 96 + 96 + 32 + 48 + 32 = 400$.

The readout MLP at kspace_conv.py:388-391:

```
self.readout = nn.Sequential(
    nn.Linear(400, 192), nn.SiLU(), nn.Dropout(0.15),
    nn.Linear(192, 192), nn.SiLU(), nn.Dropout(0.15),
    nn.Linear(192, 1),
)
```

Total: $76,992 + 37,056 + 193 = \mathbf{114,241}$ parameters.

The final `log_sigma_shift` is a single-scalar `nn.Parameter` (kspace_conv.py:393) initialised to -5.0 . Before each fold trains, the training script (08_train_hybrid.py:304) calls `KSENet.set_target_shift(mean(log_sigma[train_indices]))` to overwrite the scalar with the training-fold mean of the target. The final prediction is

$$\hat{y} = \text{readout}(\mathbf{h}) + s$$

with s holding the per-fold shift (kspace_conv.py:508-509).

A.10 Component table — verified parameter counts

This table consolidates §§A.2–A.9 and matches `sum(p.numel() for p in m.parameters() if p.requires_grad)` on the headline KSENet construction. File-line citations omit the kspace_conv.py prefix (for example, :259 means kspace_conv.py:259); items 7a–7b live in bond_valence_field.py and are flagged accordingly.

#	Component	File:line	Params	Input $\rightarrow$ output
1	Atom embedding	:259	9,600	$(N,) \rightarrow (N, 96)$
2	Structure factor	:412--426	0	$(N, 96), (N, 3) \rightarrow (B, 1241, 96) \mathbb{C}$
3	KSECBLOCK $\times 3$	:190--210	537,804	$(B, K, D) \mathbb{C} \rightarrow (B, K, D) \mathbb{C}$
3a	KubicHarmonicFilter	:79--115	30,016	filter on $(k , K_0, K_{4a}, K_{4b}, K_{6a}, K_{6b})$
3b	CrossShellGatedAttn	:118--175	148,484	4-head gated self-attention over K
3c	$2 \times$ _CLN	:178--187	768	LayerNorm($2D$) on $[\text{Re}; \text{Im}]$
4	Magpie projection	:291--296, :456--461	22,344	$(B, 132) \rightarrow (B, 96)$
5	Lattice projection	:303--308, :463--468	1,360	$(B, 8) \rightarrow (B, 32)$
6	Geometric projection	:314--319, :470--475	3,650	$(B, 25) \rightarrow (B, 48)$
7a	LearnableBVParams [†]	bvf:68--101	202	per-species (r_0, b) via softplus
7b	compute_bv_features [†]	bvf:104--185	0	$(N, 3), (B, 3, 3) \rightarrow (B, 8)$
7c	DBVF projection	:341--346, :484--493	1,360	$(B, 8) \rightarrow (B, 32)$
8	Readout MLP	:388--391, :508	114,241	$(B, 400) \rightarrow (B,)$
9	log_sigma_shift	:393, :395--397, :509	1	scalar additive shift
Total			690,562	

[†] bvf = bond_valence_field.py.

A.11 Equivariance and invariance, summarised

The forward pass enforces three structural constraints that are not learned, only respected:

1. **Permutation invariance over atoms** — all atom-level operations (embedding lookup, structure-factor sum, BV sum) are symmetric in the order of atoms within a crystal. Implemented through `index_add_` (`kpace_conv.py:422`) and the unordered double sum in DBVF (`bond_valence_field.py:158-169`).
2. **Cubic-rotation (O_h) equivariance of the k-space stream** — the wavevector grid is itself O_h -invariant (§A.4 grid construction); the filter depends on $\mathbf{k}$ only through O_h -invariant scalars; the cross-shell gate depends only on $|k|$, also invariant. So rotating a structure by any O_h rotation leaves $\mathbf{F}$ globally permuted but pointwise unchanged after the encoder, and the mean pool over K is invariant.
3. **Periodic-boundary awareness** — DBVF’s ± 1 image shell (27 shifts) covers the dominant Li–anion bonds across periodic boundaries, since the 4 Å cutoff is much shorter than typical lattice constants.

A.12 Optional / inactive components (consolidated brief)

The codebase supports several components that are *not* in the headline. They are mentioned here only so a reader of the source code is not confused.

Component	File:line	When active / status
Path Bond-Valence Field	<code>path_bv_field.py</code>	flag <code>--use-path-bv-field</code> ; Phase B2 negative result, SI.
BatteryNet dual-stream (MPNN + cross-attention bridge)	<code>mpnn_encoder.py</code> ; <code>cross_attention_bridge.py</code> ; <code>kspace_conv.py:353--369</code>	flag <code>--dual-stream</code> ; Phase B3 negative result, SI.
MACE precomputed energetics (slot)	<code>kspace_conv.py:325--334</code>	active when <code>mace_dim > 0</code> ; never populated in headline.
Heteroscedastic NLL + log-variance head (HybridHead)	<code>08_train_hybrid.py:109--160</code>	flag <code>--hetero</code> ; Phase A1 negative result, SI.
MC-dropout uncertainty	<code>kspace_conv.py:511--527</code>	invoked from <code>scripts/05_mc_dropout.py</code> ; calibration figure only, not in main MAE.

These are documented exhaustively in `RESULTS-kSEC.md`; this reference does not re-derive their forward equations.

Part B — Mechanism (mathematical formulation)

B.0 Notation reminder

All symbols defined in the front-matter glossary. We use $\mathbb{C}$ for the complex numbers, $\odot$ for elementwise multiplication, $\|\cdot\|$ for the Euclidean norm, $\mathbb{1}[\cdot]$ for an indicator, and $\sigma(\cdot)$ for the logistic sigmoid (overloaded with conductivity σ — context disambiguates).

B.1 Featurization equations

Magpie composition vector. A Magpie feature is a function of a Composition object $C = \{(z_e, n_e)\}_e$ where z_e is an element atomic number and n_e is its multiplicity in the reduced formula. Each of the 132 features is one of `{mean, mean_dev, min, max}` of one of 22 elemental scalar properties (atomic mass, atomic radius, electronegativity, etc.) over C , plus a few count-based features. Implemented in `matminer.featurizers.composition.ElementProperty.from_preset("magpie")`; this document treats it as a black-box 132-d real vector.

Lattice 8-vector. Given a `pymatgen.core.Lattice` object,

$$\text{lattice_feats} = [a, b, c, \alpha, \beta, \gamma, V, N/V] \in \mathbb{R}^8$$

where (a, b, c) are edge lengths in Å, (α, β, γ) are inter-axis angles in radians (`featurize.py:81-83` converts degrees), V is the cell volume (Å³), and N/V is the atomic number density.

Geometric 25-vector. See §A.1 table; full formulae are in `geometric.py:85-277`. The bond-valence strain proxy at index 23 is itself a non-differentiable forward of Brown 2002 — that proxy is *not* the same object as DBVF, which is differentiable.

B.2 The reciprocal-space transform

For crystal b with N_b atoms at fractional positions $\{\mathbf{r}_n\}_{n \in b}$, atomic numbers $\{z_n\}_{n \in b}$, embedding lookup $\mathbf{e}_n = E[z_n]$, and a fixed grid of K integer wavevectors $\{\mathbf{k}_m\}$:

$$\mathbf{F}_b(\mathbf{k}_m) = \frac{1}{N_b} \sum_{n \in b} \mathbf{e}_n \exp(-2\pi i \mathbf{k}_m \cdot \mathbf{r}_n) \in \mathbb{C}^D$$

This is the (atom-count-normalised) discrete Fourier transform of the embedded atomic density on a per-channel basis. Translation by $\mathbf{r}_n \rightarrow \mathbf{r}_n + \mathbf{t}$ multiplies $\mathbf{F}_b(\mathbf{k}_m)$ by a unit-magnitude phase $e^{-2\pi i \mathbf{k}_m \cdot \mathbf{t}}$ that vanishes (after the $|\cdot|$ readouts in higher layers; see §C.1) when the structure is rigidly translated as a whole.

`kspace_conv.py:412-426`.

B.3 Cubic-rotation invariants of $\hat{\mathbf{k}}$

The cubic rotation group O_h is the largest finite point group preserving the cubic Bravais lattice — it has 48 elements (24 proper rotations and their inversions). On the unit sphere of directions

$\hat{\mathbf{k}} = \mathbf{k}/|\mathbf{k}|$, the lowest-degree non-trivial O_h -invariant polynomial appears at degree 4. We use a five-element invariant basis on $\hat{\mathbf{k}} = (x, y, z)$ (`kspace_conv.py:52-76`):

$$\begin{aligned} K_0(\hat{\mathbf{k}}) &= 1 \\ K_{4a}(\hat{\mathbf{k}}) &= x^4 + y^4 + z^4 - \frac{3}{5} \\ K_{4b}(\hat{\mathbf{k}}) &= x^2y^2 + y^2z^2 + z^2x^2 - \frac{1}{5} \\ K_{6a}(\hat{\mathbf{k}}) &= x^6 + y^6 + z^6 - \frac{3}{7} \\ K_{6b}(\hat{\mathbf{k}}) &= x^2y^2z^2 - \frac{1}{105} \end{aligned}$$

The constants subtract the spherical mean so that each invariant is zero-mean on the sphere — useful as features for a downstream MLP. Each K_* is invariant under any rotation $R \in O_h$: $K_*(R\hat{\mathbf{k}}) = K_*(\hat{\mathbf{k}})$. The complete list is computed once at `kspace_conv.py:280-283` and stored as a buffer.

B.4 Kubic-harmonic filter

Let $\mathbf{H} \in \mathbb{C}^{B \times K \times D}$ be the input k-space tensor and $f_\theta : \mathbb{R}^6 \rightarrow \mathbb{R}^{2D}$, $g_\phi : \mathbb{R}^6 \rightarrow \mathbb{R}^{2D}$ the two MLPs in `KubicHarmonicFilter`. Per k-mode m :

$$\mathbf{u}_m = [|k_m|, K_0(\hat{\mathbf{k}}_m), K_{4a}(\hat{\mathbf{k}}_m), K_{4b}(\hat{\mathbf{k}}_m), K_{6a}(\hat{\mathbf{k}}_m), K_{6b}(\hat{\mathbf{k}}_m)]$$

$$\mathbf{w}_m = f_\theta(\mathbf{u}_m), \quad \mathbf{b}_m = g_\phi(\mathbf{u}_m)$$

splitting the $2D$ outputs into real-imaginary halves to form complex tensors $\mathbf{W}_m, \mathbf{B}_m \in \mathbb{C}^D$. The filter output is

$$(\text{filt}(\mathbf{H}))_{b,m,d} = \mathbf{W}_{m,d} \cdot \mathbf{H}_{b,m,d} + \mathbf{B}_{m,d}$$

`kspace_conv.py:102-115`. Crucially, $\mathbf{u}_m$ depends on $\mathbf{k}_m$ only through invariant scalars, so $\mathbf{w}_m$ and $\mathbf{b}_m$ are unchanged under any O_h rotation of the k-grid; the filter therefore commutes with O_h rotations of the input crystal.

B.5 Cross-shell gated attention

Let $\mathbf{X} = [\text{Re } \mathbf{H}; \text{Im } \mathbf{H}] \in \mathbb{R}^{B \times K \times 2D}$ and $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{C}^{B \times K \times D}$ formed by splitting `qkv(X)` into six real chunks and pairing them as `complex(real, imag)`. With h heads of dimension $d_h = D/h = 24$:

$$s_{ij}^{(b,h)} = \frac{\text{Re} \langle \mathbf{q}_i, \mathbf{k}_j^* \rangle_{(b,h)}}{\sqrt{d_h}} + \log \text{gate}^{(h)}(|k_i|, |k_j|, ||k_i| - |k_j||)$$

where $\text{gate}^{(h)}(\cdot) \in (0, 1)$ is the h -th output of a small MLP (3→32→4) followed by a logistic sigmoid (`kspace_conv.py:136-139, :157-158`). The attention weights $\alpha_{ij}^{(b,h)} = \text{softmax}_j(s_{ij}^{(b,h)})$

are applied to the value tensor:

$$\mathbf{o}_i^{(b,h)} = \sum_j \alpha_{ij}^{(b,h)} \mathbf{v}_j^{(b,h)}$$

The heads are concatenated to $\mathbf{o} \in \mathbb{C}^{B \times K \times D}$, real-imag-stacked, projected through `o : Linear(2D, 2D)` and split back to a complex tensor; finally a residual $\mathbf{H}$ is added (`kpace_conv.py:172-175`).

The Hermitian inner-product score $\text{Re} \langle \mathbf{q}, \mathbf{k}^* \rangle = \text{Re} \mathbf{q} \cdot \text{Re} \mathbf{k} + \text{Im} \mathbf{q} \cdot \text{Im} \mathbf{k}$ is real-valued, so softmax is well-defined. The gate term enters as log gate so that $\text{gate} \in (0, 1)$ is equivalent to multiplying the un-normalised exponential weight by gate; this lets the module suppress cross-shell attention for distant $|k_i|, |k_j|$ pairs without zeroing out the path entirely.

B.6 KSECBLOCK forward

`kpace_conv.py:201-210`:

$$\begin{aligned} \mathbf{H} &\leftarrow \text{CLN}_{\text{in}}(\mathbf{H}) \\ \mathbf{H} &\leftarrow \text{filt}(\mathbf{H}) \\ \mathbf{H} &\leftarrow \mathbf{H} \odot \sigma(|\mathbf{H}| - 1) \\ \mathbf{H} &\leftarrow \text{CLN}_{\text{attn}}(\mathbf{H}) \\ \mathbf{H} &\leftarrow \text{CrossShellAttn}(\mathbf{H}) \end{aligned}$$

The attention call already includes its own residual (line 175). After three blocks, the model mean-pools over the K axis and stacks real and imaginary parts (`kpace_conv.py:451-454`).

B.7 DBVF forward

`bond_valence_field.py:104-185`. Given parameters $r_0, b \in \mathbb{R}^{101}$ (softplus-reparameterised), atom species z_n , fractional positions $\mathbf{r}_n$, and the per-crystal lattice $\mathbf{C}_b \in \mathbb{R}^{3 \times 3}$:

Cartesian positions and image shifts:

$$\mathbf{x}_n = \mathbf{r}_n \cdot \mathbf{C}_b, \quad \Delta_k = \mathbf{s}_k \cdot \mathbf{C}_b, \quad \mathbf{s}_k \in \{-1, 0, 1\}^3 \text{ (27 shifts)}$$

Bond-valence sum at Li site i (cation): Let $\mathcal{A}_b = \{j : z_j > 0, z_j \neq 3\}$ be the anion set. Then

$$V_i = \sum_{j \in \mathcal{A}_b} \sum_{k=1}^{27} \exp\left(\frac{r_0(z_j) - d_{ijk}}{b(z_j)}\right) \cdot \mathbb{1}[d_{ijk} < 4 \text{ \AA}]$$

with $d_{ijk} = \|\mathbf{x}_j + \Delta_k - \mathbf{x}_i\|$. The mismatch is

$$U_i = |V_i - V_{\text{target}}|, \quad V_{\text{target}} = 1.$$

Per-crystal aggregates (`bond_valence_field.py:172-183`): for the set $\{U_i\}_{i \in \text{Li}_b}$ of all Li-site mismatches in crystal b ,

$$\text{DBVF}_b = [\mu, \sigma_U, \min, \max, p_{25}, p_{50}, p_{75}, \frac{2}{\pi} \arctan|\text{Li}_b|] \in \mathbb{R}^8$$

where the last entry is a smooth saturating count of Li sites.

Gradient path. V_i depends on r_0, b through the differentiable exponential; the indicator $\mathbb{1}[d < 4]$ has zero gradient, but the cutoff is a hard mask, not a soft sigmoid, so as long as the cutoff remains well outside the typical Li–anion bond length the optimisation does not see the discontinuity. Gradients flow back through `_inv_softplus`-initialised raw parameters with `softplus'(x) =  $\sigma(x)$` .

The (B, 8) aggregate is layer-normed and projected to (B, 32) (§A.8.3) before concatenation.

B.8 Auxiliary projection heads

The Magpie, Lattice, Geometric, and DBVF heads share a structural template

$$\text{proj}(\mathbf{x}) = \text{SiLU}(\mathbf{W}_2 \text{Drop}(\text{SiLU}(\mathbf{W}_1 \text{LayerNorm}(\mathbf{x}) + \mathbf{b}_1)) + \mathbf{b}_2)$$

(Lattice and DBVF omit the dropout layer between the two SiLUs.) This template is the same up to dimensions; differences are tabulated in §A.10.

B.9 Concatenation and readout

$$\mathbf{h} = [\text{Re } \bar{\mathbf{F}} \parallel \text{Im } \bar{\mathbf{F}} \parallel \text{proj}_{\text{Magpie}} \parallel \text{proj}_{\text{Lattice}} \parallel \text{proj}_{\text{Geometric}} \parallel \text{proj}_{\text{DBVF}}]$$

with $\mathbf{h} \in \mathbb{R}^{B \times 400}$. The readout MLP is

$$\text{readout}(\mathbf{h}) = \mathbf{W}_3 \text{Drop}(\text{SiLU}(\mathbf{W}_2 \text{Drop}(\text{SiLU}(\mathbf{W}_1 \mathbf{h} + \mathbf{b}_1)) + \mathbf{b}_2)) + b_3$$

returning a scalar per crystal.

B.10 Target shift

Before fold f trains, `08_train_hybrid.py:304` calls

$$s \leftarrow \frac{1}{|\text{train}_f|} \sum_{i \in \text{train}_f} \log_{10} \sigma_i$$

via `KSENet.set_target_shift`. The final prediction is $\hat{y}_b = \text{readout}(\mathbf{h}_b) + s$. The shift s is itself a `nn.Parameter` and continues to receive gradients during training, but the per-fold initialisation centers it near the truth.

B.11 Loss

08_train_hybrid.py:349:

$$\mathcal{L} = \frac{1}{B} \sum_{b=1}^B (\hat{y}_b - \log_{10} \sigma_b)^2$$

i.e. mean squared error on the log conductivity, single-task. The headline does **not** use heteroscedastic NLL, multi-task auxiliary losses, or Arrhenius-consistency terms (those are documented as Phase A1/A4 negative results in the SI; see §A.12).

B.12 Optimisation

08_train_hybrid.py:332-333, :351:

$$\theta_{t+1} = \text{AdamW}(\theta_t, \nabla_{\theta} \mathcal{L}; \text{lr} = 10^{-3}, \text{weight_decay} = 10^{-4})$$

with `CosineAnnealingLR(T_max = 60)` on the learning rate (one step per epoch), and gradient-norm clipping at $\|\nabla\| \leq 1.0$ (`torch.nn.utils.clip_grad_norm_`).

B.13 Cross-validation protocol

The headline is reported as the average of an ensemble of 5 random seeds $\times$ 5 stratified folds = 25 trained models. The stratification bins the eligible $\log_{10} \sigma$ targets into 10 quantile buckets (`configs/base.yaml:33`); each fold receives one tenth of each bucket, so high-conductivity samples are not all bunched into one fold. Eligibility filter: $\log_{10} \sigma > -15$ (drops 4 samples; `phaseB1.log:2`). 281 eligible samples remain.

For each (seed, fold), the training script holds out one fold’s test set, calls `set_target_shift` on the train mean (B.10), runs 60 epochs of AdamW + cosine, and saves the checkpoint with the best validation MAE. The 25 fold predictions are concatenated to form an out-of-fold (OOF) prediction vector of length 281, used both for the headline MAE 1.047 number and as one of the two inputs to the stacking layer (B.14). Per-seed and per-fold metrics are stored in `results/ksec_phaseB1.json`.

B.14 Pretraining (encoder only)

The atom embedding and the three KSECBLOCKS are pretrained on 18,574 lithium-containing crystals from the Materials Project (`scripts/12_pretrain_mp.py`) before the headline training run. The pretext task is a two-target regression on formation energy and band gap with mean-squared-error losses; only the encoder weights (67 tensors per `phaseB1.log:3`) are transferred to the downstream model — the readout, target shift, projection heads, and DBVF parameters are all newly initialised at fine-tune time.

The pretrained encoder is loaded with `KSENet.load_state_dict(..., strict=False)` so that downstream-only buffers and parameters do not block the load.

B.15 Stacking with a tabular gradient-boosted-tree baseline

`scripts/10_stacking.py`. The reported “stacked” MAE of **0.980** is computed by combining the headline OOF predictions with a parallel LightGBM + Magpie OOF baseline. The procedure:

1. **Baseline.** For each seed $\times$ fold split, fit `LightGBMRegressor(n_estimators=400, lr=0.05, num_leaves=31, min_child_samples=5)` on the 132-d Magpie composition vectors of the training fold and predict the test fold’s $\log_{10}\sigma$. Average the 5 seeds to obtain `lgb_oof` : (281,). Standalone MAE: 0.999 (`stacking_phaseB1.json:3`).
2. **Stack input.** Form the per-sample 2-vector $\mathbf{u}_b = [\hat{y}_b^{\text{kSEC}}, \hat{y}_b^{\text{LGB}}]$.
3. **Meta-learner.** Fit a 5-fold ridge regressor `Ridge(alpha=1.0)` on $\mathbf{u}$ targeting the true $\log_{10}\sigma$. Coefficients average to $(c_{\text{kSEC}}, c_{\text{LGB}}) \approx (0.56, 0.49)$ across folds, intercepts near 0.20 (`scripts/10_stacking.py:149-166, stacking_phaseB1.json` per-fold coefficients).

The stacked OOF prediction $\hat{y}_b^{\text{stack}} = c_0 \hat{y}_b^{\text{kSEC}} + c_1 \hat{y}_b^{\text{LGB}} + c_2$ achieves MAE 0.980, $R^2 = 0.637$, and AUC ≈ 0.895 for the binary “fast conductor” classification at the 10^{-4} S cm $^{-1}$ threshold.

Part C — Reasoning (design justification)

C.1 Why reciprocal space?

Three reasons motivate moving the encoder’s primary representation into Fourier space.

First, **rigid translations of the structure decouple from the magnitude representation**. A uniform shift $\mathbf{r}_n \rightarrow \mathbf{r}_n + \mathbf{t}$ multiplies $\mathbf{F}_b(\mathbf{k}_m)$ by a global phase $e^{-2\pi i \mathbf{k}_m \cdot \mathbf{t}}$ that is constant in n . After mean-pooling over K and reading out modulus-like quantities (`kspace_conv.py:451-454`), translation invariance falls out without explicit data augmentation.

Second, **periodic boundary conditions are native to reciprocal space**. The discrete Fourier transform on integer wavevectors automatically respects the lattice’s translational symmetry — no expensive per-bond image search is needed inside the encoder. Real-space crystal-graph networks (CGCNN, MEGNet, etc.) all spend a meaningful share of their forward-pass cost building periodic-image neighbour lists; k-SEC pays this cost zero times.

Third, **the structure factor is a physically natural primitive**. X-ray and neutron diffraction measure $|F(k)|^2$ directly; many properties of crystals — phonon spectra, electronic band gaps via Bloch’s theorem, charge-density-wave instabilities — are most natural in reciprocal space. Building feature maps that live in k -space throughout puts the encoder closer to physically interpretable quantities than building them in real space.

The trade-off: real-space locality is more transparent. We address this by adding the geometric and DBVF streams (which are real-space at heart) as parallel branches into the readout (§C.6).

C.2 Why O_h cubic invariants specifically?

The cubic rotation group O_h has 48 elements and is the largest finite point group of the cubic Bravais lattice. The lowest-degree non-trivial O_h -invariant polynomial on the unit sphere appears at $\ell = 4$ — that is, $\ell = 1, 2, 3$ contain no non-zero O_h -invariants. Up to $\ell = 6$, the model exposes a five-dimensional invariant basis (§B.3) which captures the leading cubic-symmetric directional dependencies a filter could plausibly need for a fast-conducting Li framework.

Two practical reasons for choosing O_h specifically:

1. **All four canonical fast-Li-ion conductor families are cubic or near-cubic at the room-temperature state.** Anti-perovskites (Li_3OCl , Li_3OBr) and chloride double-perovskites are cubic. LGPS-family electrolytes are tetragonal but very near cubic; argyrodites are cubic. The bias toward cubic-symmetric directional response is therefore well-matched to the regime the model is deployed in.
2. **Lower-symmetry crystals tolerate the O_h surplus.** A monoclinic crystal does not have O_h as a symmetry of its lattice, but it still has a well-defined Fourier transform on the integer wavevector grid, and forcing O_h equivariance on the *encoder* simply means the encoder ignores any O_h -breaking direction-dependence that would not generalise. The crystal’s true symmetry is encoded *in the data* (different lattices give different $|k|$ distributions and $\hat{\mathbf{k}}$ distributions of weight) — the encoder doesn’t have to learn to be O_h -invariant; it is, by construction, and any structure factor magnitude that depends on direction beyond the O_h -invariant scalars simply has to be expressed through $|k|$ and the orbit averages. *This last point is inferred from the architectural structure rather than spelled out in code comments.*

C.3 Why cross-shell gated attention?

A purely radial filter $W(|k|)$ — common in radial-basis networks like SchNet — would mix only k -modes within the same shell (same $|k|$). But Umklapp-style coupling, where a wave’s momentum is changed by a reciprocal lattice vector during a scattering event, is a real and important physics in crystals — especially for ionic-conductivity-relevant low-energy phonons. The cross-shell gated attention path lets the model couple different $|k|$ shells while keeping the *gate* a function of magnitudes only (so O_h -equivariance is preserved). The gate’s MLP input — $(|k_i|, |k_j|, ||k_i| - |k_j||)$ — gives it the freedom to learn either same-shell-dominant behaviour (small $|k_i| - |k_j|$ favoured) or umklapp-style cross-shell behaviour (large $\Delta|k|$ favoured) per head.

The architectural docstring at `kspace_conv.py:21-24` makes the umklapp framing explicit.

C.4 Why bond-valence as inductive bias for σ ?

Brown’s bond-valence theory (1953/2002) is a classical, empirically-fit relationship between cation–anion bond distance and an *implied* valence: short bonds contribute valence near the cation’s nominal oxidation state, longer bonds contribute exponentially less. Sites where the BVS deviates from the nominal valence are *ill-fitting* — they are over- or under-bonded relative to what the chemistry expects — and ill-fitting Li sites are precisely the sites where Li-ion mobility is high (because the local coordination is loose and the energy landscape is shallower).

DBVF takes this inductive bias and makes it learnable: the per-anion r_0, b parameters are no longer frozen at Brown’s tabulated values but are gradient-trained with the rest of the model. The architectural justification is that Brown’s review used *bulk-crystal* data to fit the parameters — but a network predicting *Li-ion conductivity in solid electrolytes* may benefit from an “effective” bond-valence parameterisation that is slightly shifted toward the parts of structure space where Li mobility is favourable. The model can learn this shift end-to-end, while remaining anchored to a known physical formula (the parameters can drift, but the functional form $\exp((r_0 - d)/b)$ does not change).

C.5 Why aggregate BV mismatch statistics, not per-site features?

DBVF returns 8 per-crystal scalars rather than per-Li-site features for two reasons. First, the downstream prediction is a *graph-level* scalar ($\log_{10} \sigma$ for the whole crystal), so per-site features would have to be pooled anyway. Second, the *distribution* of mismatches across Li sites is informative — a crystal with a few badly-fitting Li sites among many well-fitting ones (high $\max(U)$, low $\text{mean}(U)$) is qualitatively different from one with uniformly mediocre fits (high $\text{mean}(U)$, low $\max(U)$), and only the former is likely to host fast Li hopping. The eight aggregates — mean, std, percentiles, min, max, count — give the readout MLP enough shape information to discriminate these cases without committing to a particular pooling.

C.6 Why fuse Magpie / Lattice / Geometric / DBVF / k-SEC streams?

Each stream captures a physically distinct scale or quantity:

- **k-SEC (192 dim)** — long-range structural correlations through $\mathbf{F}(k)$.
- **Magpie (96 dim)** — pure composition statistics; ignorant of structure entirely.
- **Lattice (32 dim)** — the global cell parameters, capturing density and aspect-ratio.

- **Geometric (48 dim)** — local Li environment, percolation, bottleneck geometry.
- **DBVF (32 dim)** — physics-informed Li-anion bond-valence mismatches.

The redundancy across these streams is *deliberate* — no single one of them captures all the information relevant to σ , and the readout MLP’s first linear layer can downweight any stream it finds redundant. The cost of redundancy is limited (these branches add together a few tens of thousands of parameters relative to the 540k k-SEC core), while the benefit of multi-scale inputs is substantial in the small-data regime ($n = 281$) where any one stream might overfit.

C.7 Why pretrain the encoder on Materials Project crystals?

OBELiX has 281 labelled samples — too few to learn good general crystal representations from scratch. The 18,574-sample Materials Project subset (`scripts/12_pretrain_mp.py`) is two orders of magnitude larger and offers cheap pretext labels (formation energy, band gap) that force the encoder to develop structure-aware representations. The pretraining transfers 67 weight tensors (atom embedding + 3 KSECBLOCKS; `phaseB1.log:3`) into the downstream run.

The downstream-specific layers — readout, target shift, projection heads, DBVF parameters — are *not* pretrained, so they remain free to adapt to the OBELiX-specific objective without inheriting biases from the pretext task.

C.8 Why MSE loss + cosine-annealed AdamW?

These are the standard defaults for small-scale regression. **MSE** on $\log_{10}\sigma$ (rather than on σ directly) is the right scale because σ spans 11 decades on OBELiX; an MSE on σ would be dominated by the few high-conductivity samples. **AdamW** decouples weight decay from the gradient-update step, which empirically helps with regression on noisy small datasets. **Cosine annealing** reduces the learning rate smoothly to zero across the 60 training epochs, which suppresses end-of-training oscillation and is the modern default for fixed-epoch training schedules.

The headline does not use heteroscedastic NLL — that was tested as Phase A1 and produced *worse* MAE despite providing per-sample uncertainty (see SI of the manuscript).

C.9 Why stack with LightGBM?

At $n = 281$, a **tabular gradient-boosted-tree model on Magpie features is competitive**: LightGBM-only achieves MAE 0.999 alone, and 0.924 once lattice and geometric features are appended. The neural model achieves 1.047 standalone — which is lower than CGCNN’s reported 1.573 and IonPath’s prior 1.393, but still above the tabular ceiling of 0.924 at this benchmark size.

The stacking step combines $\hat{y}^{\text{kSEC}}$ and $\hat{y}^{\text{LGB}}$ via a tiny 2-coefficient ridge meta-learner. This achieves MAE 0.980 — between the two standalone numbers — which is what we’d expect if the two models capture *partially complementary* signal: the ridge weights ($\approx 0.56, 0.49$ averaged across folds) confirm that both streams contribute non-trivially. The stacked R^2 of 0.637 is materially higher than either model alone (LightGBM: 0.602, k-SEC: 0.602), confirming complementarity.

C.10 Component complementarity

What does k-SEC capture that DBVF does not, and vice versa?

- **k-SEC** captures long-range, periodic, direction-dependent structure through the structure-factor representation. It is sensitive to the *whole* arrangement of atoms — sublattice ordering, anion-framework geometry, etc. — but is agnostic to specific physics (it has to learn what is relevant).
- **DBVF** captures a single, narrowly-defined, physically-motivated quantity (Li site bond-valence mismatch). It is direct and chemically interpretable but ignores everything beyond the cation’s first coordination shell.
- **Magpie** captures composition only; it is invariant to structure entirely. It provides a strong baseline because composition correlates with chemistry and chemistry correlates with σ .
- **Lattice** and **Geometric** carry static descriptors of cell shape and Li-environment connectivity that the k-SEC encoder would otherwise have to discover from raw $\mathbf{F}(k)$.

The combination is expected to outperform any single stream because they are *partly orthogonal*: the headline ablation (Phase A vs Phase B in `RESULTS-kSEC.md`) shows that adding DBVF moves the standalone MAE from 1.103 (pre-pivot) to 1.047 — a 5 % relative improvement — and that the stacking with LightGBM improves further from 0.999 (LGB alone) to 0.980.

C.11 Honest negative result: DBVF features vs. DBVF architecture

A “brutal-honest test” reported in `RESULTS-kSEC.md:178-191`: the trained DBVF features (the 8-d per-crystal aggregates) were extracted from a fully-trained Phase B1 model and fed to LightGBM as additional features, alongside Magpie + Lattice + Geometric. The result: **MAE 0.933 (worse than the 0.924 baseline of LightGBM + Magpie + Lattice + Geometric alone)**.

This rules out a feature-engineering interpretation of DBVF’s contribution. The architectural value of DBVF is *not* in the 8-d aggregate feature vector itself; it is in **gradient flow through r_0 and b during end-to-end training**. The aggregates by themselves are weak descriptors. They become useful only when the rest of the network is co-trained with the DBVF parameters and learns to consume them. This is the strongest single piece of evidence that DBVF’s contribution is *architectural, not feature-engineering*.

Verification

How to confirm the documented forward path

Smoke tests (pytest tests/test_smoke.py from the repo root). Five tests cover:

1. Wyckoff wavevector generation and Fourier basis evaluation (test_wyckoff_wavevectors).
2. CanonicalRecord round-trip (data schema).
3. CIF $\rightarrow$ CrystalGraph parsing on a synthetic Li_2O .
4. KSENet forward and backward on a small synthetic batch (test_ksec_forward_backward).
5. Translation invariance under uniform fractional shifts (test_ksec_equivariance_under_translation)
— directly validates §A.11 / §C.1.

```
cd c:/Users/sunwo/Desktop/battery
pytest tests/test_smoke.py
```

Reproduce the standalone MAE 1.047 without retraining: the OOF predictions are committed at results/ksec_phaseB1_oof.npz. The number is the seed_ensemble.mae field of results/ksec_phaseB1.json.

Reproduce the stacked MAE 0.980 without retraining:

```
python scripts/10_stacking.py \
    --ksec-oof results/ksec_phaseB1_oof.npz \
    --results /tmp/stacking_check.json
```

The output JSON's seed_ensemble.mae field will read 0.980 (within float-precision drift).

Verify parameter counts in §A.10: instantiate the model and sum:

```
import sys; sys.path.insert(0, "src")
from ionpath.models.kspace_conv import KSENet
m = KSENet(num_species=100, feature_dim=96, num_blocks=3, n_max=2,
           tabular_dim=132, lattice_dim=8, geometric_dim=25,
           bv_field=True, bv_mobile_z=3)
print(sum(p.numel() for p in m.parameters() if p.requires_grad))
# → 690562
```

What changes (and what doesn't) when flags toggle

--use-bv-field --use-lattice --use-geometric --use-magpie (default for Magpie via tabular_dim=132) reproduces the 690,562-parameter headline. Removing any of those flags reduces both the parameter count (by the corresponding row of §A.10) and the dimension of **h** at the readout (by the corresponding *_hidden).

Flags that activate components *not* in the headline (--dual-stream, --use-path-bv-field, --hetero, --use-mace) are documented in §A.12; their forward equations are not in this reference.

References (key code paths)

Citation	Where
Brown 2002 bond-valence parameters	<code>src/ionpath/models/bond_valence_field.py</code> :45-54
Cubic-rotation invariants	<code>src/ionpath/models/kspace_conv.py</code> :52-76
KSECNet forward	<code>src/ionpath/models/kspace_conv.py</code> :399-509
Wyckoff wavevector generation	<code>src/ionpath/utils/wyckoff_fourier.py</code> :75-103
27-image bond-valence sum	<code>src/ionpath/models/bond_valence_field.py</code> :104-185
Training loop	<code>scripts/08_train_hybrid.py</code> :336-367
Stacking (LightGBM + ridge)	<code>scripts/10_stacking.py</code> :48-169
Pretraining pretext	<code>scripts/12_pretrain_mp.py</code> (formation energy + band gap)
Headline run log	<code>phaseB1.log</code>
Headline metrics	<code>results/ksec_phaseB1.json</code>
Honest brutal-test (DBVF features → LightGBM)	<code>RESULTS-kSEC.md</code> :178-191